

run1-5.sh

You need to understand how run1-5.sh works to make your own script.

Learning Goals

- Explain what `run1-5.sh` adds to `run1-4.sh` : Sanctum, users, and the token table.
- Distinguish a migration file from a migrated table.
- Read how the script skips existing tables and seeds demo users safely.

What run1-5.sh Sets Up

1. Prepare: Create the Laravel project (only once) and install Sanctum
2. Check: Is MySQL running?
3. Database: Create studentdb and the ase230 user if missing
4. Configure: Point Laravel's .env at studentdb
5. Copy: Copy the lesson files into the project
6. Migrate: Create the `students` , `users` , and token tables if missing
7. Seed: Insert sample students and two demo users
8. Verify: Print the API routes

`run1-5.sh` is almost the same as `run1-4.sh` .

- Steps 1 to 4 are identical. See `4-2. run1-4.sh.md` , including the WSL2 `sudo mysql` login.
- The new parts are in Steps 5 to 7.

```
cd "5. Bearer Token APIs"
bash ./run1-5.sh
cd student-api
php artisan serve --port=8080
# WSL2: php artisan serve --host=0.0.0.0 --port=8080
```

What `run1-5.sh` Adds to `run1-4.sh`

Step	New in this lesson
Copy	<code>AuthController.php</code> , <code>User.php</code> (with <code>HasApiTokens</code>), <code>UserSeeder.php</code>
Migrate	<code>users</code> and <code>personal_access_tokens</code> , one file at a time with <code>--path</code> ; a table that already exists is skipped
Seed	Two demo users

Email	Password
<code>student@university.edu</code>	<code>student123</code>
<code>teacher@university.edu</code>	<code>teacher456</code>

Step 5: Copy the Lesson Files

File	Role
Api/AuthController.php	login , logout
Api/StudentController.php	Student (reads and public)
Models/Student.php	Same model Lesson 4
Models/User.php	Generate model pl HasApi trait

Migration File vs. Table

A migration file only describes a table. The table exists after you **run** the migration.

	<code>run1-4.sh</code>	<code>run1-5.sh</code>
<code>install:api</code>	Yes	Yes
<code>personal_access_tokens</code> migration file	Created	Created
Migration run	No: only <code>students</code>	Yes: <code>students</code> , <code>us</code> <code>personal_access_</code>
Table in <code>studentdb</code>	Not created	Created
		<code>createToken()</code> on

- `run1-4.sh` runs one migration with `--path`, so the token file just sits in `database/migrations/`.
- `run1-5.sh` runs the token migration on purpose, because login now needs the table.
- Check the result: `mysql -u ase230 -p studentdb -e "SHOW TABLES"`.

```
+-----+
| Tables_in_studentdb |
+-----+
| cache                |
| cache_locks          |
| failed_jobs          |
| job_batches          |
| jobs                 |
| migrations           |
| password_reset_tokens |
| personal_access_tokens |
| sessions             |
| students             |
| users                |
+-----+
```

<--

Step 6: Skip a Table That Already Exists

`studentdb` may already have these tables from an earlier full
`php artisan migrate`.

- Each project has its own migration file with a different timestamp.
- Laravel sees a new file, tries to create the table again, and fails with `1050 Table already exists`.

```
table_exists() {  
    MYSQL_PWD="$DB_PASSWORD" mysql -h 127.0.0.1 -u "$DB_USERNAME" -N \  
        -e "SHOW TABLES LIKE '$1'" "$DB_DATABASE" | grep -q .  
}  
  
table_exists users || php artisan migrate --path=... --force  
table_exists personal_access_tokens || php artisan migrate --path=... --force
```

Syntax	Meaning
<code>SHOW TABLES LIKE 'users'</code>	Prints the table name if it exists, nothing if it does not
<code>grep -q .</code>	Succeeds if there was any output at all
<code>a b</code>	Run <code>b</code> only if <code>a</code> fails : here, only if the table is missing
<code>-N</code>	Do not print the column header

- The students migration has its own `Schema::hasTable()` check, so it always runs.

Step 7: Seed and Verify

```
php artisan db:seed --class=StudentSeeder --force  
php artisan db:seed --class=UserSeeder --force  
php artisan route:list --path=api
```

- `UserSeeder` uses `User::firstOrCreate()`, so rerunning does not add duplicate users.
- The `User` model hashes the password automatically (`'password' => 'hashed'`). The seeder passes plain text, and the database stores a hash.